REPRESENTAÇÃO DE CONHECIMENTO

Luís Morgado

ISEL-DEI

LINGUAGENS LÓGICAS

Linguagens formais ou semiformais que utilizam estruturas e regras simbólicas para representar conhecimento com base em lógica formal

- Programação declarativa
  - **O que se sabe** acerca do problema a resolver
  - **Inferência** a partir do que se sabe
- Representação de conhecimento
  - **Factos**
  - **Regras**
- Exemplo:
  - **Linguagem *Prolog***

LINGUAGEM PROLOG

- ***Programming in Logic***
- Linguagem de programação **simbólica**
- Baseada na lógica de predicados
- Facilita a programação através da redução da complexidade de especificação
- Propicia a realização de programas claros, concisos e compreensíveis

LINGUAGEM PROLOG

- **Mecanismos base**
  - *Pattern matching* (mecanismo de unificação)
  - *Backtracking*
- **Motor de inferência**
  - Máquina virtual
    - WAM (*Warren Abstract Machine*)
- **Foco da programação**
  - O que se sabe acerca do problema a resolver
  - Dedução automática de informação a partir do que se sabe (inferência)

LINGUAGEM PROLOG

Exemplo:

Representação e processamento de informação genealógica

Factos conhecidos:

- O João é pai do Rui
- A Rita é mãe do Carlos
- A Rita é mãe do João
- O Rui é pai do José
- O José é pai do António
- O Carlos é pai do Joaquim

LINGUAGEM PROLOG

Exemplo:

Representação e processamento de informação genealógica

Factos conhecidos:

- O João é **pai** do Rui
- A Rita é **mãe** do Carlos
- A Rita é **mãe** do João
- O Rui é **pai** do José
- O José é **pai** do António
- O Carlos é **pai** do Joaquim

Especificação em Prolog:

```
pai(joao, rui).
mae(rita, carlos).
mae(rita, joao).
pai(rui, jose).
pai(jose, antonio).
pai(carlos, joaquim).
```

LINGUAGEM PROLOG

Exemplo:

Representação e processamento de informação genealógica

Relações conhecidas:

- Pai: x é pai de y
- Mãe: x é mãe y
- Filho: x é filho de y
- Irmão: x é irmão de y
- Avô: x é avô de y
- Tio: x é tio de y

Representação sob a forma de *predicados*:

```
pai(X, Y)
mae(X, Y)
filho(X, Y)
irmao(X, Y)
avo(X, Y)
tio(X, Y)
```

LINGUAGEM PROLOG

Especificação em Prolog

Regras:

```
filho(X, Y) :- pai(Y, X).
filho(X, Y) :- mae(Y, X).

irmao(X, Y) :-
   filho(X, Z),
   filho(Y, Z),
   X \= Y.

avo(X, Y) :-
   filho(Z, X),
   filho(Y, Z).

tio(X, Y) :-
   filho(Y, Z),
   irmao(X, Z).
```

Interrogação:

```
?- tio(joao, X).

X = joaquim
```

TIPOS DE REPRESENTAÇÃO

Procedimental (*Imperativo*)

- Representação **procedimental** de *como* obter um resultado específico: ***"Saber como…"***
- Controlo representado explicitamente
- Exemplos de linguagens: *C*, *C++*, *Java*

Declarativo (*Denotacional*)

- Representação **declarativa** de *o que* se sabe acerca de um domínio: ***"Saber que…"***
- Controlo não representado explicitamente
- Exemplo de linguagem: *Prolog*

EXEMPLO

Cálculo de $f(n) = n!$

Linguagem Python: *Especificação imperativa*

```
1. def factorial(n):
2.     if not n >= 0:
3.         raise ValueError("n tem de ser >= 0")
4.     resultado = 1
5.     factor = n
6.     while factor > 1:
7.         resultado *= factor
8.         factor -= 1
9.     return resultado
```

EXEMPLO

Cálculo de $f(n) = n!$

Linguagem Prolog: *Especificação declarativa*

```
1. factorial(0, 1).
2. factorial(N, F) :-
3.     N > 0,
4.     N1 is N - 1,
5.     factorial(N1, F1),
6.     F is N * F1.
```

LINGUAGEM PROLOG

Programa

- Um programa em linguagem *Prolog* consiste na definição de **relações lógicas** (*predicados*) e da **interrogação** acerca dessas relações

Elementos de um programa:

- **Predicados**
  - Definem relações entre objectos/conceitos
- **Cláusulas**
  - Expressões simbólicas que representam o que se conhece acerca de um determinado domínio
  - Um predicado é definido por uma ou mais cláusulas

LINGUAGEM PROLOG

Predicados

Definem relações entre objectos/conceitos

- Representam propriedades que atribuem valores de verdade a tuplos de valores
- Caracterizados por:
  - **Nome**
  - **Aridade** (número de argumentos)

Exemplo:

`chove`	Predicado de aridade = 0 (*proposição*)
`pai(X,Y)`	Predicado de aridade = 2

LINGUAGEM PROLOG

Cláusulas

Expressões simbólicas que indicam o que se conhece acerca de um determinado domínio, podem assumir a forma de ***factos*** ou de ***regras***

As cláusulas são especificadas como frases e podem incluir conectivas lógicas (operadores)

- **Conectivas lógicas** (operadores)
  - Conjunção (*e*): **a, b**
  - Disjunção (*ou*): **a; b**
- **Cláusulas especificadas como frases**
  - "." termina cláusula

LINGUAGEM PROLOG

Factos

Cláusulas que representam conhecimento que é considerado verdadeiro

Exemplo:

```
chove.
```

"*É verdade que chove*"

```
pai(joao, rita).
```

"*O João é pai da Rita*"

LINGUAGEM PROLOG

Regras

Cláusulas que definem relações lógicas entre *conclusões* e *condições*

- Utilizadas para inferir conhecimento a partir de factos conhecidos
  - Inferência dedutiva (**a** *se* **b**)
- **Sintaxe e semântica**:

  a :- b

  "**a** *é verdade se* **b** *é verdade*" (**b** ⟶ **a**)

Exemplo:

```
tio(X, Y) :- filho(Y, Z), irmao(X, Z).
```

LINGUAGEM PROLOG

Interrogações

Cláusulas que permitem conhecer se uma determinada conclusão (relação ou facto) é verdadeira

- **Sintaxe e semântica**:

  ?- b

  "**b** *é verdade?*" ou "*É verdade que* **b***?*"

Exemplo:

```
?- pai(joao, X).
```

$(\exists x)(\text{pai}(\text{joao}, x))$?

LINGUAGEM PROLOG

Interrogação

- Sequência de um ou mais ***objectivos***
- A resposta a uma interrogação consiste na *satisfação* dos *objectivos*
- ***Satisfação*** de um objectivo
  - Demonstração de que o objectivo é verdadeiro, com base nas relações definidas
    - O objectivo é uma consequência lógica dos factos e das regras definidas
- Se a interrogação incluir variáveis, são encontradas as atribuições de valor para as quais os objectivos são satisfeitos

LINGUAGEM PROLOG

Termos (estruturas simbólicas)

Representam dados ou estruturas de dados

Podem assumir várias formas:

- **Átomos**
  - Símbolos com nome iniciado por uma letra minúscula ou entre aspas simples, por exemplo:
    `arvore`, `'Árvore'`
- **Números**
  - Número inteiros ou reais, por exemplo:
    `0`, `1.23`
- **Variáveis**
  - Símbolos com nome iniciado por uma letra maiúscula ou pelo caracter "_", por exemplo:
    `X`, `Nome`, `_a001`

LINGUAGEM PROLOG

- **Variáveis**
  - Símbolos que referenciam outros símbolos
    - Primeira letra maiúscula ou "_"
  - Atribuição de valor por substituição
    - Mecanismo de *unificação*
  - **Exemplo:**

```
?- X = abc, Y = X, write(Y).
abc

?- X = abc, Y = X, writeln(Y), fail; writeln(Y).
abc
_G5540
```

LINGUAGEM PROLOG

Mecanismo de unificação

- Determinação da substituição mais geral que iguale dois termos, se existir
  - Unificação de padrões simbólicos ("*pattern matching*")

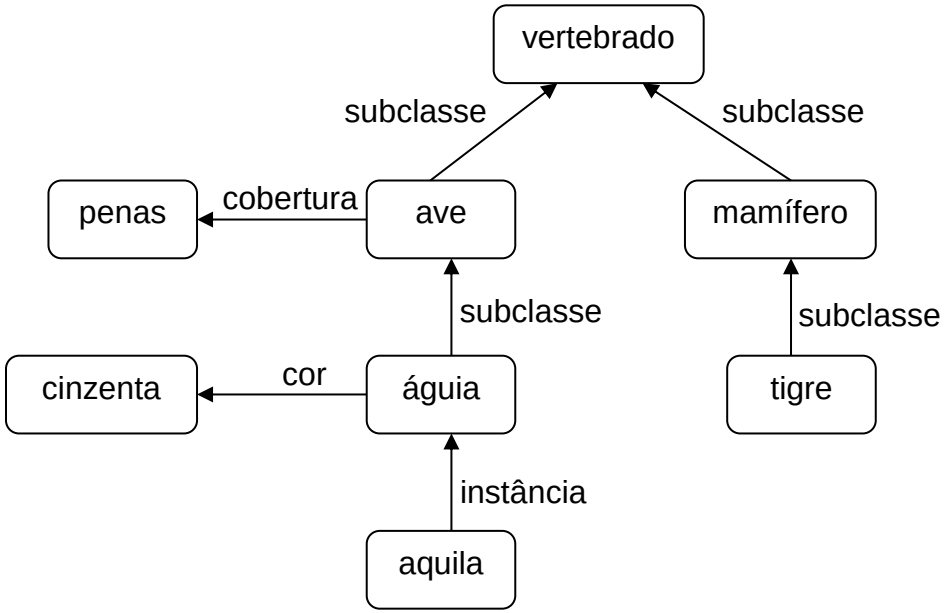
Exemplo:

```
?- X + 2 * Y = 3 + Z * 4.
X = 3,
Y = 4,
Z = 2.

?- data(Dia, Mes, Ano) = data(1, janeiro, 2000).
Dia = 1,
Mes = janeiro,
Ano = 2000.
```

CONCRETIZAÇÃO

Realizar em linguagem *Prolog* a representação da seguinte rede semântica:



Testar as seguintes interrogações (entre outras):

```
?- caracteristica(aquila, cobertura, penas).
?- caracteristica(X, cobertura, penas).
```

LINGUAGEM PROLOG

Programação simbólica

- Programação baseada na manipulação de estruturas simbólicas
  - **Exemplo**:
    - **Computação numérica (*Python*)**

```
>>> X = 2 * 3 + 4 - 5; print(X)
5
```

    - **Computação simbólica (*Prolog*)**

```
?- X = 2 * 3 + 4 - 5, write(X).
2 * 3 + 4 - 5
```

- **Significado (semântica) definido por:**
  - Relações entre símbolos
  - Relações dos símbolos com o domínio do problema

LINGUAGEM PROLOG

Computação simbólica *vs.* numérica

- Operador **=**
  - Representa a unificação simbólica das expressões
- Operador ***is***
  - Requer a avaliação numérica das expressões para unificação

Exemplo:

Factorial simbólico

```
factorial(0, 1).
factorial(N, F) :-
   N > 0,
   N1 = N - 1,
   factorial(N1, F1),
   F is N * F1.

?- factorial(4, F).
F = 4*(3*(2*(1*1)))
```

Factorial numérico

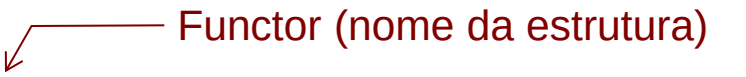
```
factorial(0, 1).
factorial(N, F) :-
   N > 0,
   N1 is N - 1,
   factorial(N1, F1),
   F is N * F1.

?- factorial(4, F).
F = 24
```

LINGUAGEM PROLOG

Estruturas de dados

- Estruturas simbólicas gerais
  - **Exemplo**:




```
Data = data(1, 2, 2003)
Data = data(dia(1), mes(2), ano(2003))
Data = 1-2-2003
```

- Listas
  - Estruturas de dados com encadeamento sequencial
  - **Exemplo**:

Definição por enumeração dos elementos da lista: [*Elemento1*, *Elemento2*, …]

```
L = [1, 2, 3]
```

Definição por encadeamento de listas: [*Cabeça da lista* | *Resto da lista*]

```
? - [1, 2, 3] = [X | Resto]
X = 1,
Resto = [2, 3].
```

LINGUAGEM PROLOG

Semântica declarativa (denotacional)

- Representação de ***o que se sabe*** acerca de um domínio de problema
  - Consiste num conjunto de afirmações descritivas acerca do domínio do problema
  - Definida pelas *relações* especificadas sob a forma de cláusulas de um programa

Exemplo:

```
pai(joao, rui).

mae(rita, carlos).

filho(X, Y) :- pai(Y, X).

filho(X, Y) :- mae(Y, X).
```

LINGUAGEM PROLOG

Semântica procedimental (operacional)

- Representação de ***como obter um resultado específico*** num determinado domínio de problema
  - Define a ordem (sequência de passos) de execução na resolução de um problema
  - Definida pela ordem das cláusulas

Exemplo:

Ordem das cláusulas define semântica procedimental

```
pai(joao, rui).

mae(rita, carlos).

filho(X, Y) :- pai(Y, X).

filho(X, Y) :- mae(Y, X).
```



```
?- filho(X, Y)
X = rui
Y = joao ;
X = carlos
Y = rita
```

LINGUAGEM PROLOG

Semântica declarativa *vs.* procedimental

- A capacidade de inferência associada à linguagem *Prolog* possibilita a resolução automática dos aspectos procedimentais de um problema
  - Facilita o foco na definição do que se sabe acerca do problema (*significado declarativo*), independentemente do procedimento específico para a sua resolução (*significado procedimental*)
  - Contribui para uma redução significativa da complexidade de resolução de um problema
- Para tirar partido desta capacidade, o foco da especificação de um programa deve ser no significado declarativo evitando, sempre que possível, o comprometimento com aspectos procedimentais

LINGUAGEM PROLOG

Predicado negação: *not*, \+

- Negação por falha

Exemplo:

```
not(P) :- P, !, fail.
not(_).

diferente(X,Y) :- not(X = Y).
```

Notação alternativa (*SWI-Prolog*): `\+`

```
diferente(X,Y) :- \+ (X = Y).
```

LINGUAGEM PROLOG

Iteração

- **Por recursividade**
  - Semântica declarativa
- **Por *backtracking***
  - Semântica procedimental
  - A sua utilização deve ser criteriosa

Exemplo:

```
factorial(0, 1).
factorial(N, F) :-
   N > 0,
   N1 is N - 1,
   factorial(N1, F1),
   F is N * F1.
```

```
repete.
repete :- repete.

teste :-
    repete,
        read(X),
        writeln(X),
        X = 'terminar'.
```

LINGUAGEM PROLOG

Iteração por recursividade

- **Casos particulares**
  - Cláusulas de finalização da iteração
- **Casos gerais**
  - Cláusulas que requerem definição recursiva

Exemplo:

```
factorial(0, 1).
```

Caso particular

```
factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.
```

Caso geral

LINGUAGEM PROLOG

Manipulação da base de dados

- Predicados que possibilitam a manipulação da base de dados interna, onde são mantidas as relações definidas
  - *assert*, *asserta*, *assertz*, *retract*, *retractall*

Exemplo:

```
?- assert(animal(tartaruga)).
?- assert(animal(baleia)).

?- animal(X), writeln(X), fail; true.
tartaruga
baleia

?- retract(animal(baleia)).

?- animal(X), writeln(X), fail; true.
tartaruga
```

REFERÊNCIAS

[Russel & Norvig, 2022]
S. Russell, P. Norvig, “Artificial Intelligence: A Modern Approach”, 4th Global Edition, Prentice Hall, 2022

[Bratko, 1986]
I. Bratko, “Prolog Programming for Artificial Intelligence”, Addison-Wesley, 1986.